COMSATS University Islamabad
Attock Campus

Final Year Project Documentation

for

E-Assistant: AI-Powered Product Search Agent

by

Amman Zarkash
FA22-BAI-019
Muhammad Umar
FA22-BAI-041

Supervised by
Dr. Zahoor Tanooli

Bachelor of Science in Artificial Intelligence
Department of Computer Science

2022–2026

Spring 2026

E-Assistant: AI-Powered Product Search Agent

This project documentation is submitted to the Department of Computer Science,
COMSATS University Islamabad, Attock Campus, in partial fulfillment of the requirements for the degree of BS in Artificial Intelligence.

Name	Registration Number
Amman Zarkash	FA22-BAI-019
Muhammad Umar	FA22-BAI-041

Supervisor

Dr. Zahoor Tanooli
Department of Computer Science
COMSATS University Islamabad, Attock Campus

April 2026

Final Approval

This project titled

E-Assistant: AI-Powered Product Search Agent

submitted by

Amman Zarkash (FA22-BAI-019)
Muhammad Umar (FA22-BAI-041)

has been reviewed and approved for submission to COMSATS University Islamabad, Attock Campus.

External Examiner: _____

Supervisor: _____

Dr. Zahoor Tanooli

Head of Department: _____

Head of Department

Author's Declaration

We, Amman Zarkash (FA22-BAI-019) and Muhammad Umar (FA22-BAI-041), declare that the work presented in this documentation is our own work carried out during the scheduled final year project period for the project titled E-Assistant: AI-Powered Product Search Agent. Wherever the words, ideas, figures, or findings of other authors have been used, they have been appropriately acknowledged through references. To the best of our knowledge, this document does not contain plagiarized material beyond the acceptable academic limit.

Student Signature: _____

Student Signature: _____

Date: _____

Certificate

It is certified that Amman Zarkash (FA22-BAI-019) and Muhammad Umar (FA22-BAI-041) have completed the work described in this report under my supervision at the Department of Computer Science, COMSATS University Islamabad, Attock Campus. In my opinion, the work fulfills the academic requirements for submission of the BS Artificial Intelligence final year project documentation.

Supervisor: _____

Dr. Zahoor Tanooli
Department of Computer Science
COMSATS University Islamabad, Attock Campus

Head of Department: _____

Dedicated to our parents, teachers, and all those
who supported us throughout this project journey.

ACKNOWLEDGEMENTS

We are grateful to Allah Almighty for the guidance, patience, and opportunity to complete this work. We sincerely thank our supervisor, Dr. Zahoor Tanooli, for academic guidance, technical feedback, and continuous support throughout the design, implementation, and documentation of this project. The completion of this system required work across software engineering, machine learning, data processing, and interface design, and that work benefited directly from disciplined supervision.

We also acknowledge the support of the Department of Computer Science and COMSATS University Islamabad, Attock Campus, for providing an environment in which a practical AI product could be developed as more than a classroom prototype. The project integrates web scraping, ranking, conversational orchestration, database persistence, and frontend interaction into one coherent system, and that breadth reflects the training received during the degree program.

We are thankful to the open-source ecosystem as well. The implementation builds on several mature tools and frameworks, including FastAPI, MongoDB, React, Vite, Playwright, Beautiful Soup, Gradio, and sentence-transformer-based retrieval utilities. These technologies made it possible to focus on system integration, product behavior, and evaluation rather than reimplementing infrastructure from scratch.

Finally, we thank our families and friends for their patience during the long phases of experimentation, debugging, interface refinement, and report writing. Their support made it possible to carry the project from an idea into a working AI-assisted product-search platform.

Project Summary

E-Assistant: AI-Powered Product Search Agent is an end-to-end AI product-search system designed to help a user search e-commerce offers through natural-language interaction and receive ranked product suggestions with follow-up support. The project combines indexed product retrieval, semantic matching, clustering, value-oriented ranking, live web fallback search, conversation memory, and a user-facing React interface under the single product identity E-Assistant.

The implementation is organized around four major runtime parts. First, a React/Vite frontend provides the user experience for search, result browsing, follow-up conversation, history restoration, and an informational about page. Second, a FastAPI backend exposes search, recommendation, assistant, interaction-logging, metrics, and labeling endpoints. Third, an assistant orchestration layer manages conversational intent, context continuity, clarification handling, comparison flow, refinement flow, and selected-product follow-ups. Fourth, a data and ranking layer stores normalized offers in MongoDB and processes them through text retrieval, semantic matching, clustering, reranking, and collaborative-filtering-based personalization.

The assistant does not act as a free-text chatbot alone. It uses an internal MCP-style tool server that exposes operational tools such as indexed offer search, live web product search, direct page inspection, offer detail retrieval, and interaction logging. This makes the system traceable: conversational responses are grounded in tool results rather than invented store data. When indexed results are weak or follow-up information is missing, the system can inspect product pages and search live web sources before answering.

The resulting system demonstrates how an AI assistant can be embedded in a realistic commerce workflow rather than presented as an isolated language model demo. It is not only a search engine and not only a chat interface. It is a multi-component product-search application with data ingestion, ranking, conversation management, persistence, and interface support. This documentation explains that implemented system in terms of requirements, architecture, data design, UI behavior, runtime algorithms, and deployment.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

This chapter introduces E-Assistant: AI-Powered Product Search Agent as a complete AI product rather than a standalone search script or model demo. The project addresses the practical problem of finding relevant, high-value products across e-commerce sources while preserving conversational continuity, traceability, and deployment realism. It combines indexed retrieval, live web support, ranking logic, interface workflows, and persistent conversation state into one implemented system.

1.1 Introduction

Online product search has a familiar weakness: the user can easily find thousands of listings, but turning those listings into a reliable decision still requires manual effort. A typical buyer must jump between stores, compare prices, inspect ratings, check availability, verify delivery conditions, and remember earlier results while refining the search. The practical problem is therefore not only retrieval. It is decision support under noisy, fragmented, and constantly changing e-commerce information.

This project addresses that problem through a conversational product-search assistant named E-Assistant. Instead of limiting the system to keyword search, the platform accepts natural-language requests such as product type, price ceiling, rating preference, or value-oriented intent. The backend interprets those requests, retrieves candidates from normalized offer data, ranks them through semantic matching and value-focused scoring, and returns grounded results to the frontend. The user can then continue with follow-up questions such as asking for delivery details, reviews, comparisons, or cheaper alternatives without restarting the interaction from the beginning.

The project is implemented as a multi-layer system. The React frontend provides the search interface, history page, and assistant-style result presentation. The FastAPI backend provides operational APIs for search, recommendation, assistant reasoning, in-

teraction logging, diagnostics, and labeling support. The assistant layer manages intent planning and tool orchestration. A MongoDB-backed data layer stores indexed offers, user interactions, and conversation state. Together, these parts turn a product-query problem into a practical AI application.

Another important characteristic of the project is that the assistant is grounded in tools. Rather than generating answers from language modeling alone, it uses structured tool calls for indexed offer search, live web product search, product-page inspection, offer-detail retrieval, and interaction logging. This tool-based design is central to the system's reliability because it allows the response to remain attached to evidence from stored offers or inspected web pages.

In summary, the project is best understood as a conversational commerce support system built around search, ranking, persistence, and grounded assistant behavior. The rest of this document formalizes that system through problem definition, requirements, architecture, design models, data design, UI coverage, and deployment structure.

1.2 Significance of the Proposed System

The significance of this project lies in both its application domain and its engineering form. From an application perspective, product discovery in local e-commerce settings is often fragmented. Different stores expose different prices, review signals, and content quality. A user therefore needs not just retrieval, but aggregation, ranking, and guided follow-up support. A system that can combine those tasks into one workflow reduces search friction and improves decision quality.

From an engineering perspective, the project is significant because it moves beyond a single-model demonstration. Many student AI systems stop at classification, retrieval, or recommendation output. This project instead carries the work into a usable product stack that includes scrapers, data normalization, ranking, a FastAPI service, a conversation-aware assistant layer, a MongoDB-backed memory store, and a React interface. That makes the project suitable for evaluation as a realistic AI software system rather than only an algorithm exercise.

The project is also educationally significant because it integrates concepts from software engineering, machine learning, information retrieval, recommender systems, web technologies, databases, and human-computer interaction. Query parsing, text relevance, clustering, ranking, collaborative filtering, rate limiting, API design, session persistence, and UI state management all contribute directly to the final application behavior.

Finally, the project is significant because it is honest about implementation trade-offs. The assistant is grounded in actual tool outputs, but the about page includes broader product-style messaging than the operational backend currently enforces. The indexed

store coverage is useful, but still limited to the configured sources and the quality of the ingestion pipeline. These tradeoffs are normal in a real AI product and make the project academically valuable because they expose the boundary between implemented capability and future expansion.

1.3 Problem Statement

Users searching for products online face three recurring difficulties. First, product information is distributed across different stores with inconsistent formatting, prices, and review evidence. Second, simple keyword search does not preserve intent well when the user adds conversational follow-ups such as "compare these two," "show cheaper ones," or "what is the delivery charge for this product." Third, a language-only chatbot can sound helpful while still inventing facts unless its answers are tied to reliable store or page evidence.

Therefore, the core problem addressed in this project is how to build a grounded AI assistant that can search indexed product offers, retrieve and rank relevant results, preserve conversational context, inspect product pages when needed, and present the output through a usable frontend workflow. The solution must support both initial search and multi-turn refinement while remaining structured enough to be deployed as a software system rather than a single script.

1.4 Scope and Objectives

The scope of the project covers the implemented path from product ingestion to user-facing interaction. On the data side, the project includes offer scraping, normalization, indexed storage, semantic matching, clustering, value-oriented ranking, and interaction logging. On the assistant side, it includes conversation memory, selected-product follow-up handling, comparison handling, refinement of prior results, clarification prompts, and tool-based grounding. On the application side, it includes a React/Vite frontend, a FastAPI backend, MongoDB persistence, and lightweight deployment scripts.

The scope does not include secure multi-user account management, production-scale distributed crawling, payment integration, or enterprise-grade catalog synchronization. The project also does not claim complete store coverage. It focuses on a practical subset of indexed sources combined with live web fallback support and page inspection when direct indexed data is insufficient.

The main objectives of this project are as follows:

- To provide a conversational product-search interface that accepts natural-language

product requests and follow-up queries.

- To aggregate and normalize offers from configured e-commerce sources into a searchable MongoDB-backed collection.
- To rank search results through a combination of query parsing, text retrieval, semantic matching, clustering, and value-oriented reranking.
- To support grounded assistant behavior through an internal MCP-style tool server for search, live web lookup, page inspection, and detail retrieval.
- To preserve conversation context through persistent chat sessions and turn history.
- To support lightweight personalization through interaction logging and collaborative-filtering-based score blending.
- To package the system as a runnable product with a React frontend, FastAPI backend, and local or Docker-based deployment path.

1.5 System Limitations/Constraints

The implemented system has several practical constraints that should be stated clearly.

First, indexed results depend on the quality and freshness of scraped and normalized offers. The search pipeline can only rank what has been collected and stored. If a store page changes structure or a source becomes stale, result quality can decline until the scraping and normalization flow is rerun successfully.

Second, the assistant can inspect product pages and search live web sources, but that does not guarantee perfect product-page extraction. The page-inspection logic explicitly recognizes that some links may resolve to listing pages rather than reliable product-detail pages. In such cases, the system avoids over-trusting extracted signals.

Third, the platform currently uses a lightweight user identity approach in the frontend based on locally stored browser IDs rather than full registered-user accounts. This is sufficient for session persistence and interaction logging in a project setting, but it is not equivalent to a production authentication system.

Fourth, collaborative filtering is present as an optional score component, but its usefulness depends on the availability and quality of interaction data and trained artifacts. Personalization is therefore a supportive feature rather than the sole ranking basis.

Fifth, the about page contains product-style marketing content that is broader than the exact operational guarantees of the backend. The documentation in this report is based on the implemented runtime behavior, not on promotional page copy.

1.6 Project Deliverables

The project produced the following practical deliverables:

- FastAPI backend: An API service with health, search, recommendation, assistant, interaction, labeling, metrics, and debug endpoints.
- Assistant orchestration layer: A conversational runtime that supports new search, follow-up handling, comparison, refinement, clarification, and context memory.
- Internal MCP-style tool server: Tool interfaces for indexed offer search, live web product search, product-page inspection, offer-detail lookup, interaction logging, and interaction reporting.
- Search and ranking pipeline: Query parsing, candidate retrieval, relevance filtering, semantic matching, clustering, reranking, and optional CF blending.
- MongoDB-backed persistence layer: Collections for normalized offers, interactions, audit logs, and conversation sessions and turns.
- React frontend: Search, dashboard-style result browsing, conversation history, and informational about page.
- Data ingestion utilities: Scrapers, processing jobs, and training jobs for model artifacts and data preparation.
- Supplementary Gradio interface: An additional lightweight entry point for testing or demonstration.
- Deployment support: A local all-in-one runner and Docker Compose configuration.

1.7 Relevance to Course Modules

This project draws directly from multiple course areas in a BS Artificial Intelligence program.

Artificial Intelligence and Machine Learning: The project uses semantic matching, ranking logic, model registry handling, and optional collaborative filtering to support decision-oriented product search.

Information Retrieval and Data Mining: Query parsing, candidate retrieval, relevance filtering, clustering, and ranked result production are all retrieval-centered tasks.

Software Engineering: The system is structured as a multi-component product with clear module boundaries, service interfaces, runtime orchestration, and deployment scripts.

Database Systems: MongoDB is used as the persistent layer for offers, chat sessions, chat turns, interactions, labeling data, and audit logs.

Web Technologies and Human Computer Interaction: The React frontend provides the visible interaction model, while the FastAPI backend exposes the service-level contract consumed by the interface.

Project Methodology: The work also involves problem formulation, system trade-offs, implementation constraints, and evaluation-driven documentation, all of which are core FYP concerns.

1.8 Research Objective

The central objective of this project is to operationalize product search and recommendation as a grounded conversational system. Concretely, the goal is to build an assistant that can take natural-language shopping requests, search stored offers, fall back to live web search when necessary, inspect product pages for evidence, maintain conversational context, and present ranked product suggestions through a usable interface.

This objective has both a retrieval side and a product side. The retrieval side concerns how to find and rank good offers. The product side concerns how to keep the conversation coherent, explainable, and usable over multiple turns.

1.9 Research Questions

The project is guided by the following practical questions:

1. How effectively can indexed product search be combined with semantic matching and ranking to return useful value-oriented product suggestions?
2. How can an assistant preserve conversational continuity when the user asks follow-up questions about an earlier result set?
3. How can tool-based grounding reduce unsupported answers in conversational product search?
4. What lightweight personalization benefits can be gained from interaction logging and collaborative-filtering-based score blending?
5. What architecture is suitable for combining product ingestion, ranking, assistant orchestration, persistence, and frontend interaction in one FYP-scale system?

1.10 Contribution

The main contributions of the project are listed below.

- It converts product search from a static keyword lookup into a grounded conversational workflow.
- It combines indexed store data with live web fallback and page inspection rather than relying on only one retrieval path.
- It introduces an internal MCP-style tool abstraction that makes assistant behavior operationally structured and auditable.
- It preserves session-level memory through MongoDB-backed conversation storage and context summarization.
- It integrates interaction logging and collaborative filtering into the search pipeline without giving up interpretable ranking behavior.
- It delivers the capability through a full stack that includes scraping, ranking, APIs, UI, and deployment support.

1.11 Chapter Summary

This chapter established the context of the project by explaining the product-search problem, the project scope, the objectives, and the delivered system boundaries. It clarified that the implemented system is a grounded conversational search platform with ranking, persistence, and interface support. The next chapter positions the project against the broader technical ideas that inform its design.

Chapter 2

Literature Review

This chapter reviews the main technical ideas behind conversational product search, retrieval and ranking, recommendation support, and web-based catalog integration. The aim is not to repeat the full literature on recommender systems or search engines, but to position the implemented system against the core concepts it uses in practice.

2.1 Conversational Search and Query Understanding

Conversational search systems try to reduce the friction of repeatedly reformulating a query. Instead of asking the user to start over every time a new constraint appears, the system preserves context and interprets the next message as either a refinement, a follow-up, a comparison request, or a completely new search. This is directly relevant to the present project because many shopping interactions are iterative. A user may begin with "Samsung phone under 50000" and then continue with "show cheaper ones," "compare the first and third," or "what is the delivery charge for this one."

The implemented project reflects this literature direction by separating initial search from conversational intent planning. The assistant explicitly distinguishes between a new search, refinement of previous results, selected-product follow-up, comparison of prior results, and clarification-needed states. This is a stronger design than treating every incoming message as a fresh standalone query.

2.2 Semantic Retrieval and Product Matching

Modern retrieval systems increasingly rely on semantic representations rather than exact keyword overlap alone. Sentence-level embeddings help a query align with product titles and descriptions even when phrasing differs. This is particularly useful in commerce because users and sellers often describe the same product differently. Semantic retrieval

therefore improves recall and robustness over raw token matching alone.

The project follows this direction through a sentence-transformer-based product matcher. The search pipeline first retrieves textual candidates from MongoDB, then applies semantic query-to-candidate matching, and finally clusters close product variants before ranking. This layered approach allows the system to keep the efficiency of indexed retrieval while improving semantic quality through learned representations [1].

2.3 Ranking, Recommendation, and Value Orientation

Retrieval alone is not enough in a shopping context. The user usually wants not just relevant products, but strong value products. That means ranking must account for more than text similarity. Price, rating, review evidence, stock status, source quality, and freshness all matter. Recommendation systems often incorporate user behavior as well, since a user's previous interactions may reveal consistent brand or price preferences.

The implemented project adopts that engineering view. It computes ranking from semantic match score, price-related information, review signals, freshness, and source effects, then optionally blends in collaborative-filtering scores when interaction data and model artifacts are available. This is not a pure recommendation system and not a pure search system. It is a hybrid ranked search workflow with personalization support.

2.4 Web Extraction and Live Product Verification

Static indexed data has an unavoidable weakness: it can become stale or incomplete. Web extraction and page verification therefore remain important when the indexed result set is weak or when the user asks for details that were not already stored. Product-page inspection is especially useful for extracting availability, delivery, warranty, and review signals that may not have been preserved in the normalized listing record.

The project operationalizes this idea through two mechanisms. First, it supports live web product search when local indexed results are sparse or when explicitly requested. Second, it supports page-level inspection with structured parsing and safety checks that avoid over-trusting listing pages. That design choice is important because it gives the assistant a grounded way to answer follow-up questions rather than relying on generated guesses [6, 7].

2.5 Related System Analysis

Table 2.1 compares common system directions against the implemented product.

Table 2.1: Related system analysis with implemented product-search solution.

System Direction	Weakness	Implemented Product Response
Keyword-only product search	Misses semantic variation and usually loses conversational context.	Adds semantic matching, follow-up reasoning, comparison, and refinement support.
Recommendation-only systems	Can personalize well, but may not answer explicit user constraints transparently.	Keeps retrieval and explicit query constraints at the center while blending personalization lightly.
Chatbot without tool grounding	Risks unsupported claims about price, rating, or availability.	Grounds responses through indexed search, product-page inspection, and live web search tools.
Static catalog search only	Cannot react well to stale or incomplete stored data.	Adds live web fallback and page inspection for freshness-sensitive questions.
Search engine without memory	Forces the user to restate the product context repeatedly.	Persists sessions, turns, and active result context in MongoDB-backed conversation memory.
Scraper-only datasets	Useful for storage, but not enough for user-facing product decision workflows.	Packages data ingestion inside a full backend, assistant, and frontend system.

2.6 Chapter Summary

This chapter explained the main technical ideas that shaped the project: conversational search, semantic retrieval, ranking and recommendation support, and live verification through web inspection. The implemented system sits at the intersection of these ideas by combining indexed search, assistant reasoning, and UI delivery. The next chapter translates that implementation into formal system requirements.

Chapter 3

Requirement Analysis

This chapter describes the implemented system in terms of user classes, functional requirements, non-functional expectations, and external interfaces. The requirements are derived from the actual backend endpoints, assistant behavior, MongoDB persistence, and React frontend flows observed in the codebase.

3.1 User Classes and Characteristics

The system serves multiple user roles even though the visible interface is lightweight.

Table 3.1: User classes and characteristics.

User Class	Typical Goal	Characteristics
End User / Shopper	Find relevant products and continue with follow-up questions.	Uses the search page, expects ranked results, asks conversational refinements, and benefits from persistent browser-level session identity.
Returning User	Resume a prior search thread.	Uses the history page to restore earlier conversation sessions and continue from previous results.
Project Demonstrator / Evaluator	Assess the system during viva, review, or classroom presentation.	Focuses on traceability, ranking quality, follow-up behavior, and visible end-to-end integration.
Developer / Maintainer	Run services, inspect data, retrain models, and update scrapers.	Understands the pipeline, Mongo collections, model artifacts, local environment variables, and deployment scripts.
Admin / Labeling Operator	Monitor health, collect metrics, and label product pairs.	Uses protected endpoints such as metrics, health details, and match-pair labeling routes.

3.2 Requirement Identifying Technique

Requirements for this project were identified through four direct techniques.

Source-code inspection: The backend routes, assistant planner, tool server, search pipeline, and frontend pages were reviewed directly to determine what the system actually supports at runtime.

Workflow tracing: The message flow from UI query submission to assistant response and result rendering was reconstructed from code, which was necessary for the activity, sequence, and state models.

Data-model inspection: Settings, schemas, and Mongo-backed stores were reviewed to understand the persistent entities that the application depends on.

UI screenshot verification: The screenshots in the documentation folder were used to validate the implemented screen flow rather than documenting hypothetical screens.

3.3 Functional Requirements

The functional requirements below describe what the delivered system must do.

Table 3.2: Functional requirements of the implemented product-search system.

ID	Requirement	Rationale
FR-1	The system shall provide a main search interface where the user can submit a natural-language product query.	Implemented by the React home page and the ‘/assistant‘ request flow.
FR-2	The backend shall support indexed search over normalized product offers with configurable ‘top_k‘.	Implemented by ‘/search‘, ‘/recommend‘, and the ‘search_offers‘ tool.
FR-3	The search pipeline shall parse user constraints such as price and rating preference before ranking results.	Implemented by query parsing and enriched effective query construction.
FR-4	The assistant shall preserve conversation context through persistent conversation IDs and stored turns.	Implemented by ‘ConversationMemoryStore‘ and the conversation endpoints.
FR-5	The assistant shall distinguish between a new search, follow-up question, comparison request, refinement request, and clarification-needed state.	Implemented by turn planning and validation inside ‘AssistantAgent‘.
FR-6	The system shall support selected-product follow-ups such as price, specs, delivery, warranty, availability, and review queries.	Implemented by product-follow-up and review-follow-up handlers.

ID	Requirement	Rationale
FR-7	The system shall support comparison between previously returned products.	Implemented by comparison resolution and compare-follow-up handling.
FR-8	The system shall support refinement of prior results using new user constraints such as cheaper, rating threshold, source filter, or price limit.	Implemented by refinement logic over prior result sets.
FR-9	The assistant shall use grounded tools rather than inventing product facts.	Enforced by the assistant prompt and MCP-style tool server.
FR-10	The system shall provide live web product search when indexed results are weak or when live coverage is needed.	Implemented by the ‘search_web_products‘ tool.
FR-11	The system shall inspect product pages to extract rating, review, availability, price, shipping, and warranty signals when needed.	Implemented by the ‘inspect_product_page‘ tool.
FR-12	The system shall expose conversation history retrieval and deletion endpoints.	Implemented by ‘/assistant/conversations/conversation_id‘ GET and DELETE routes.
FR-13	The frontend shall preserve recent sessions in browser storage and allow the user to resume them.	Implemented by ‘sessionStore‘ and the history page.
FR-14	The system shall log user interactions such as view, click, save, and purchase for personalization support.	Implemented by the ‘/interactions‘ route and ‘log_interaction‘ tool.

ID	Requirement	Rationale
FR-15	The backend shall provide health, detailed health, metrics, debug, and labeling interfaces for operational support.	Implemented by ‘/health‘, ‘/health/details‘, ‘/metrics‘, ‘/debug/llm‘, and labeling routes.
FR-16	The project shall provide both local and container-based startup paths.	Implemented by ‘run_all.py‘, ‘docker-compose.yml‘, and the API/web Docker setup.

3.4 Non-Functional Requirements

Non-functional requirements describe how the system should behave operationally.

3.4.1 Reliability

- NFR-R1: The API should reject invalid requests clearly rather than failing silently.
- NFR-R2: Conversation state should survive across requests through MongoDB-backed session persistence.
- NFR-R3: The system should continue to operate even when LLM parsing fails by falling back to deterministic parsing.
- NFR-R4: Product-page inspection should avoid trusting oversized, invalid, or non-HTML responses.

3.4.2 Usability

- NFR-U1: The main search workflow should remain visible and usable without forcing the user through setup steps.
- NFR-U2: The interface should expose both conversation text and a product result panel for quick comparison.
- NFR-U3: The user should be able to resume prior sessions through a dedicated history screen.
- NFR-U4: Tool activity should remain visible through an expandable work-log component when tracing is enabled.

3.4.3 Performance

- NFR-P1: Ranked search should operate on bounded candidate sets to avoid unbounded clustering cost.
- NFR-P2: The system should reuse loaded models and registry-selected artifacts instead of reloading them on every request.
- NFR-P3: Live search and page inspection should remain bounded by response-size, timeout, and redirect limits.
- NFR-P4: The frontend should render results incrementally and preserve responsive interaction during conversation flow.

3.4.4 Security and Control

- NFR-S1: Protected endpoints should support API-key or JWT-based authorization according to configuration.
- NFR-S2: Rate limiting should be enforceable through in-memory or Redis-backed strategies.
- NFR-S3: Product-page inspection should avoid private-network access when configured to do so.
- NFR-S4: Admin-only operational routes should remain separated from normal write operations.

3.5 External Interface Requirements

The project depends on several interface boundaries: user-to-frontend interaction, frontend-to-backend HTTP calls, backend-to-database persistence, assistant-to-tool calls, and optional backend-to-web inspection.

3.5.1 User Interface Requirements

- The system shall provide a search page with a chat area, input control, quick prompts, and a result panel.
- The system shall provide a history page for recent conversation sessions and restoration actions.

- The system shall provide an about page that summarizes the product concept and feature highlights.
- The system shall provide top-level navigation between Search, History, and About routes.
- The system shall present product cards with price, rating, and source information when results are available.

3.5.2 Software Interfaces

- React, Vite, Framer Motion, and React Router for the frontend interface and navigation.
- FastAPI, Uvicorn, and Pydantic for the service layer.
- MongoDB and PyMongo for persistence.
- Sentence-transformer-based matching and collaborative filtering utilities for ranking support.
- Playwright, Requests, and Beautiful Soup for source scraping and page inspection support.
- Gradio as a supplementary lightweight interaction layer.

3.5.3 Hardware Interfaces

- General-purpose CPU execution for API, ranking, and frontend operation.
- Persistent storage for MongoDB data and project artifacts.
- Network access for local frontend-to-backend communication and optional web inspection.

3.5.4 Communication Interfaces

- HTTP-based JSON communication between the React frontend and FastAPI backend.
- MongoDB connection over the configured database URI.
- External HTTP requests for page inspection and live web lookup where enabled.
- Metrics exposition through the Prometheus-compatible ‘/metrics‘ endpoint.

3.6 Chapter Summary

This chapter documented what the project must do and how it must behave to satisfy its implementation goals. It formalized the conversational search workflow, the persistence requirements, the tool-grounded assistant behavior, and the external interface structure. The next chapter translates those requirements into architecture, diagrams, data design, and deployment detail.

Chapter 4

Design and Architecture

This chapter explains how the implemented project is organized internally. It defines the major modules, shows the overall architecture, and presents runtime diagrams for workflow, actors, classes, sequences, and states. It also documents the data design, UI flow, algorithms, tools, and deployment environment.

4.1 Modules

The system is organized into cooperating modules that separate ingestion, ranking, assistant reasoning, persistence, UI behavior, and deployment.

Data Ingestion and Source Collection: This module covers the scraper implementations and configuration-driven source collection. It produces raw product offers from Daraz, iShopping, Shophive, and other configured or live sources, which later become part of the searchable product corpus.

Offer Normalization and Searchable Storage: This module transforms raw offers into normalized MongoDB records, prepares searchable fields, and stores the data in collections such as ‘offers_normalized‘ and related catalog structures.

Search and Ranking Pipeline: This module parses the query, retrieves indexed candidates, filters relevance, applies semantic matching, clusters duplicate-like offers, reranks them with value-aware logic, and optionally blends CF scores.

Assistant Orchestration and Conversation Memory: This module handles turn planning, follow-up resolution, clarification, comparison, refinement, session persistence, and assistant response synthesis. It is the core of the conversational behavior.

MCP-Style Tool Service and Live Verification: This module exposes operational tools for offer search, live web search, product-page inspection, offer-detail retrieval, interaction logging, and interaction reporting.

Frontend Experience and Session UX: This module provides the React interface

for search, result visualization, work-log expansion, history restoration, and informational navigation.

Deployment and Operations: This module includes local orchestration, Docker deployment, rate limiting, metrics, audit logging, and health diagnostics.

4.2 Architectural Design

The project follows a layered application design with a conversational runtime at its center. At the presentation layer, the React frontend handles search submission, result rendering, recent-session storage, and navigation across Search, History, and About pages. The frontend does not rank products locally. It calls backend APIs and presents returned data in a split interaction-and-results layout.

Behind the frontend sits the FastAPI service layer. This layer exposes endpoints for search, recommendation, assistant interaction, conversation history, deletion, interaction logging, labeling, metrics, and health inspection. The API layer is also responsible for request validation, authorization enforcement, rate limiting, and audit logging.

The assistant layer is a separate logical tier inside the backend. It plans each turn, decides whether the request is a new search, follow-up, comparison, refinement, clarification, or general context question, and then uses the internal tool server to gather grounded evidence. This layer also updates conversation memory and session state after each assistant response.

The tool and ranking layer sits below the assistant. Indexed search is handled by the ‘SearchPipeline‘, which uses MongoDB-backed offer collections, semantic matching, clustering, reranking, and optional collaborative filtering. Live product-page verification and web search are handled by the MCP-style tool server. Persistent state is stored in MongoDB across offers, interactions, audit logs, sessions, and turns. The result is a modular system in which conversational behavior, ranking logic, and persistence are separated but tightly integrated.

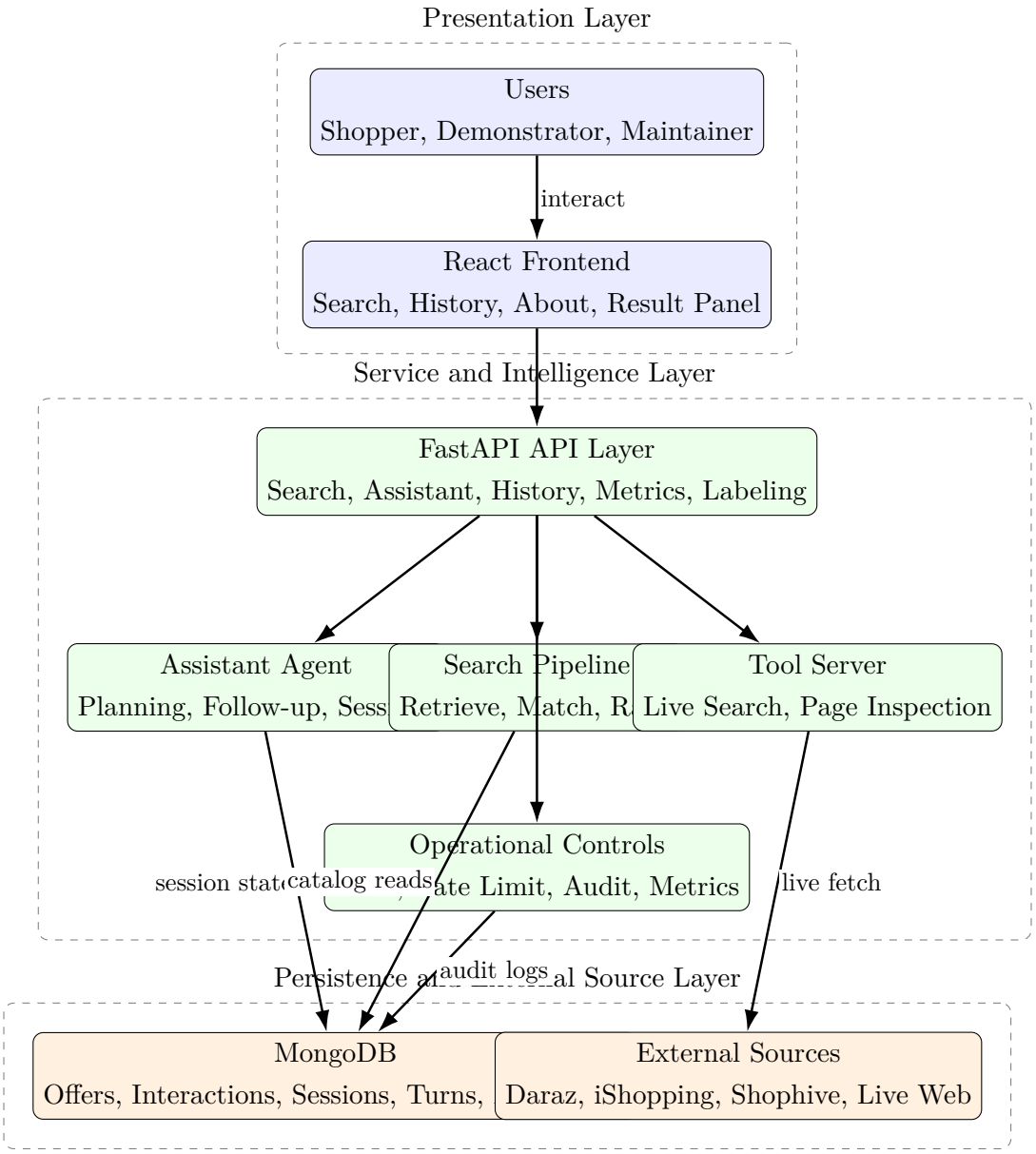


Figure 4.1: High-level architecture of the implemented product-search system.

4.3 Design Models

The following models summarize the runtime behavior and structure of the system. Each diagram is kept portrait-oriented for readability in printed and PDF form.

4.3.1 Activity Diagram

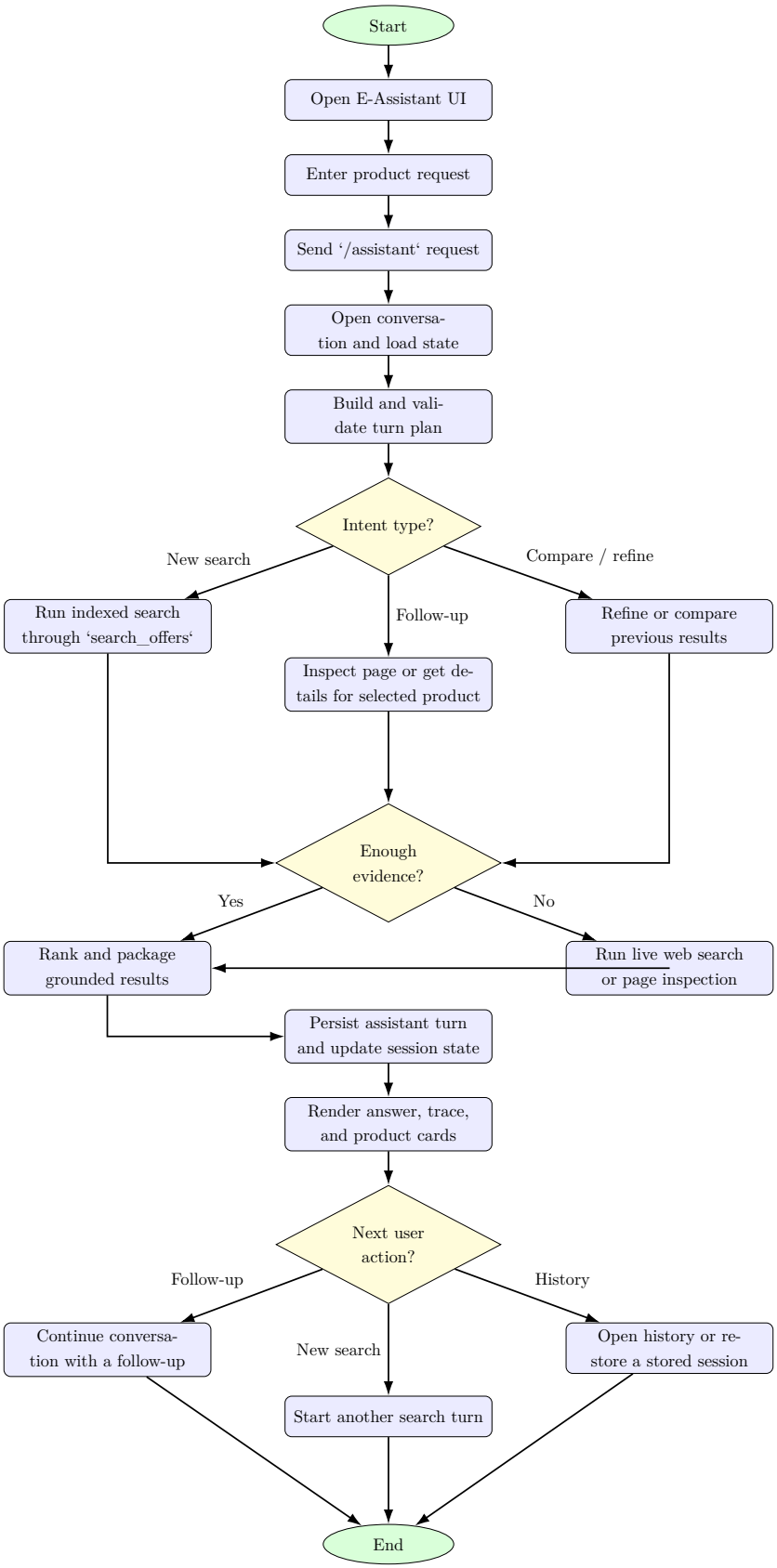


Figure 4.2: Activity diagram for conversational product search and follow-up handling.

4.3.2 Usecase Diagram

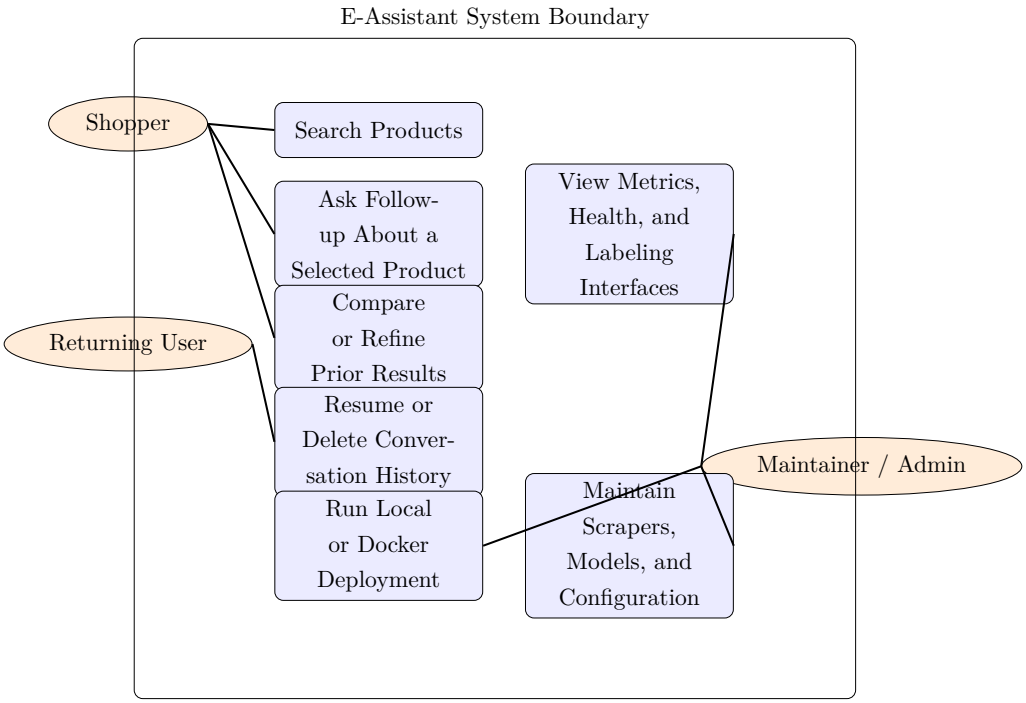


Figure 4.3: Use case diagram of the implemented product-search system.

4.3.3 Class Diagram

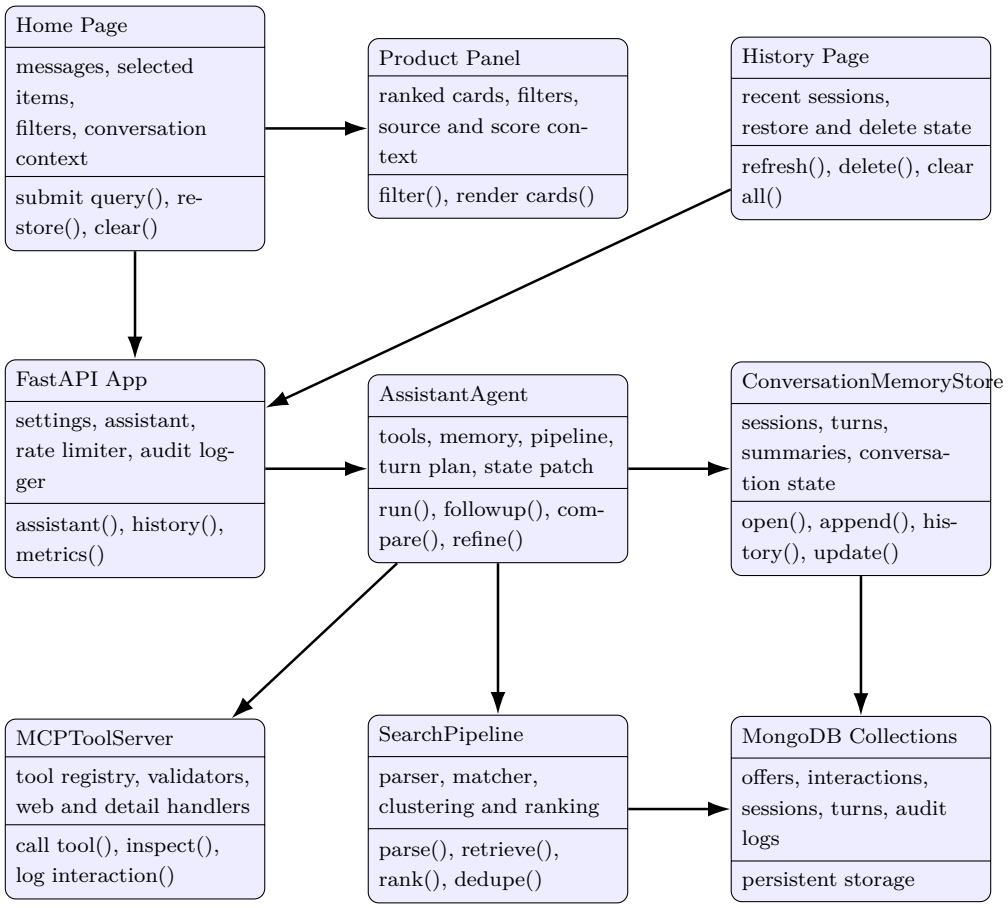


Figure 4.4: Conceptual class diagram for the frontend, backend, assistant, and persistence layers.

4.3.4 Sequence Diagram

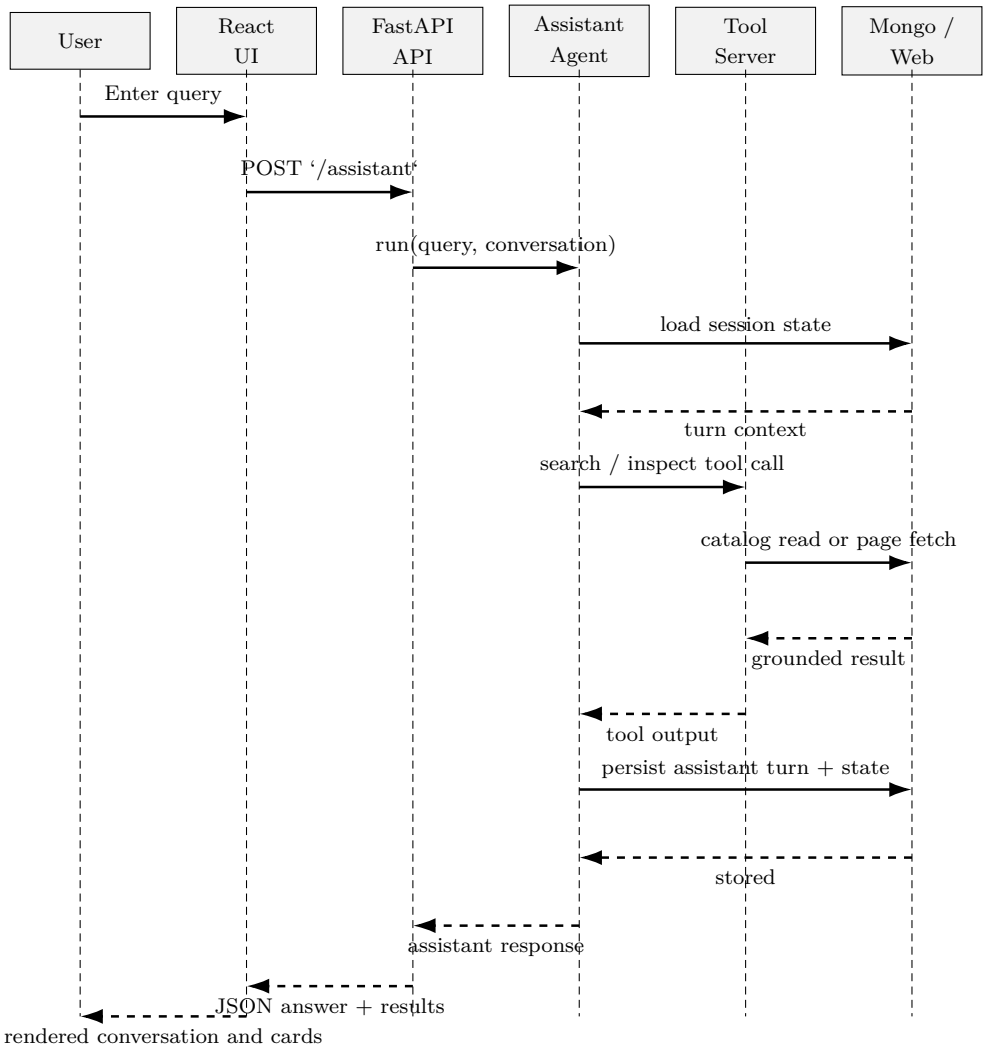


Figure 4.5: Sequence diagram for an end-to-end assistant search request.

4.3.5 State Transition Diagram

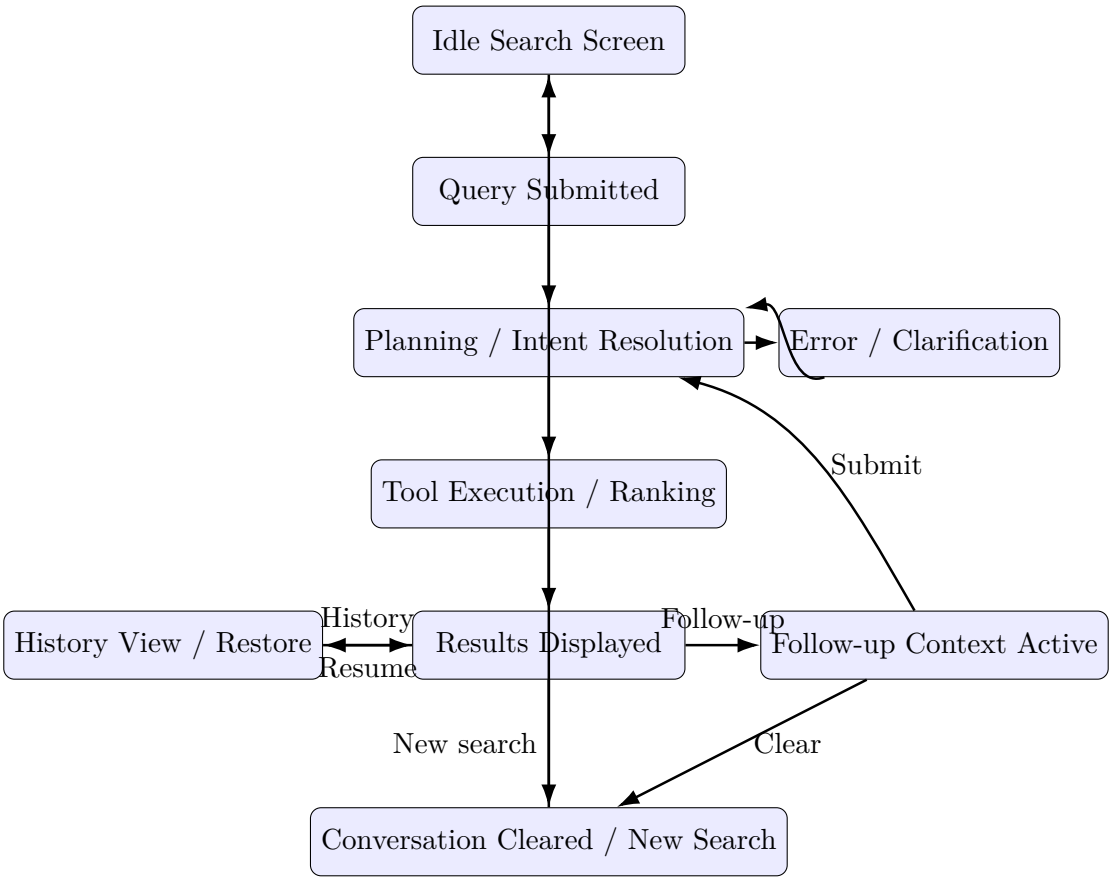


Figure 4.6: State transition diagram for the frontend and assistant interaction flow.

4.4 Data Design

The project uses both persistent and transient data.

Persistent data is stored in MongoDB collections. These include raw offers, normalized offers, canonical products, price history, labeled match pairs, user interactions, API audit logs, assistant tool logs, conversation sessions, and conversation turns. Persistent data is what allows the system to operate across runs, preserve user context, and support later training or diagnostics.

Transient data is created during request handling. This includes parsed queries, candidate sets, semantic scores, cluster labels, assistant plans, tool outputs, live web search results, page-inspection outputs, and frontend-rendered result states. These runtime structures are essential to the assistant flow even when they are not permanently stored in full.

4.4.1 Data Dictionary

Tables 4.1 and 4.2 summarize the main stored entities and request/response elements.

Table 4.1: Data dictionary for main persistent collections and entities.

Entity	Type	Description
offers_raw	Collection	Raw source listings produced by scraper modules before normalization and ranking use.
offers_normalized	Collection	Searchable normalized product offers with source, price, rating, specifications, stock, and related fields.
canonical_products	Collection	Higher-level grouped product entities that support cross-offer consolidation.
match_pairs_labeled	Collection	Product-pair records used for product-matching evaluation and labeling workflows.
user_interactions	Collection	Logged events such as view, click, save, and purchase used for personalization support.
chat_sessions	Collection	Session-level records storing conversation ownership, state, summary, expiry, and next sequence.
chat_turns	Collection	Turn-level conversation records storing role, content, metadata, timestamp, and expiry.
api_audit_logs	Collection	Request audit records that capture method, path, status code, IP, latency, and rate-limit outcome.
assistant_tool_logs	Collection / log target	Operational log target for assistant tool activity and debugging information.
Model Registry Artifacts	Filesystem	Active matcher and CF model references selected through the model registry directory.

Table 4.2: Data dictionary for assistant and search requests/responses.

Field	Type	Description
Query	String	Natural-language request submitted by the user through search or assistant endpoints.
Conversation ID	String	Stable identifier for a conversation session used to preserve and restore context.
User ID	String	Browser-level or service-provided user identity used for history and interaction ownership.
Top K	Integer	Result-count limit used in search and assistant requests.
Minimum Rating	Numeric	Optional rating threshold used to bias or filter ranked offers.
Assistant Intent	String	Planned assistant action such as new search, refinement, follow-up, comparison, or clarification.
Assistant Context	Structured data	UI-oriented context describing mode label, selected offer, comparison set, and decision reason.
Tool Trace	Structured data	Optional list of tool calls and high-level outputs returned for frontend inspection.
Ranked Result	Structured data	Offer record containing title, link, source, price, rating, review count, ranking signals, and reason.
Interaction Event	Structured data	Event record containing user, offer, source, link, event type, weight, and timestamp.
Page Inspection Output	Structured data	Extracted signals from a live product page such as rating, review count, shipping, warranty, and availability.
Work Log Entry	Structured data	UI-facing summary of a backend tool invocation shown in the expandable work-log component.

4.5 Human Interface Design

The human interface design is based on a clear split between conversation and product evidence. The user does not interact with a generic blank chatbot alone. Instead, the UI keeps search, result ranking, and assistant context visible at the same time. This is an important design choice because shopping support requires both explanation and comparable product cards.

The search page starts with a lightweight welcome state, source-status pills, quick prompts, and a single input field. Once the user submits a query, the conversation pane shows the assistant's grounded answer while the result panel displays product cards, ranking badges, filters, and context chips. This allows the system to support both question answering and browsing without forcing the user to switch between separate screens for every step.

The history page is intentionally simple. It shows recent sessions, timestamps, result counts, and resume/delete actions. This matches the actual browser-storage-based session model used in the implementation. The page does not present complex account settings because the current system is not built around full authenticated user accounts.

The about page works as an informational product surface. It explains the platform idea, feature themes, and value proposition. In architectural terms it is secondary to the search workflow, but it is useful for demonstration because it gives evaluators a quick overview of what the platform is intended to do.

4.6 Algorithm

The runtime logic can be described in two related algorithms: indexed conversational search and selected-product follow-up.

Indexed conversational search algorithm

1. Receive a natural-language request at the frontend and send it to the ‘/assistant‘ endpoint with the current conversation ID and user ID.
2. Open or create the conversation session and load recent turns and stored state from MongoDB.
3. Build a turn plan that classifies the request as a new search, refinement, comparison, selected-product follow-up, clarification-needed state, or general question.
4. If the request is a new search, parse the query and retrieve indexed candidates from ‘offers_normalized‘ using text search and filters.

5. Apply relevance filtering, semantic query-to-candidate matching, clustering, value-oriented reranking, and optional collaborative-filtering score blending.
6. If indexed coverage is weak or if live evidence is required, call live web search and/or page inspection tools.
7. Build a grounded response and product result set from tool outputs rather than from unsupported generated content.
8. Persist the assistant turn, compact result preview, assistant context, and updated session state in MongoDB.
9. Return the answer, results, and optional tool trace to the frontend for rendering.

Selected-product follow-up algorithm

1. Detect that the current user message refers to an earlier result or an active selected product.
2. Resolve the referenced product from stored conversation state or prior result indexes.
3. Retrieve saved normalized offer details and inspect the product page when delivery, availability, warranty, specs, price, or review evidence is needed.
4. Merge trusted stored and inspected signals while rejecting obvious listing-page misreads or price outliers.
5. Generate a grounded answer focused on the requested aspect, such as delivery, review strength, or comparison verdict.
6. Persist the new assistant turn and update the active product state for the next follow-up.

4.7 Tools and Technologies

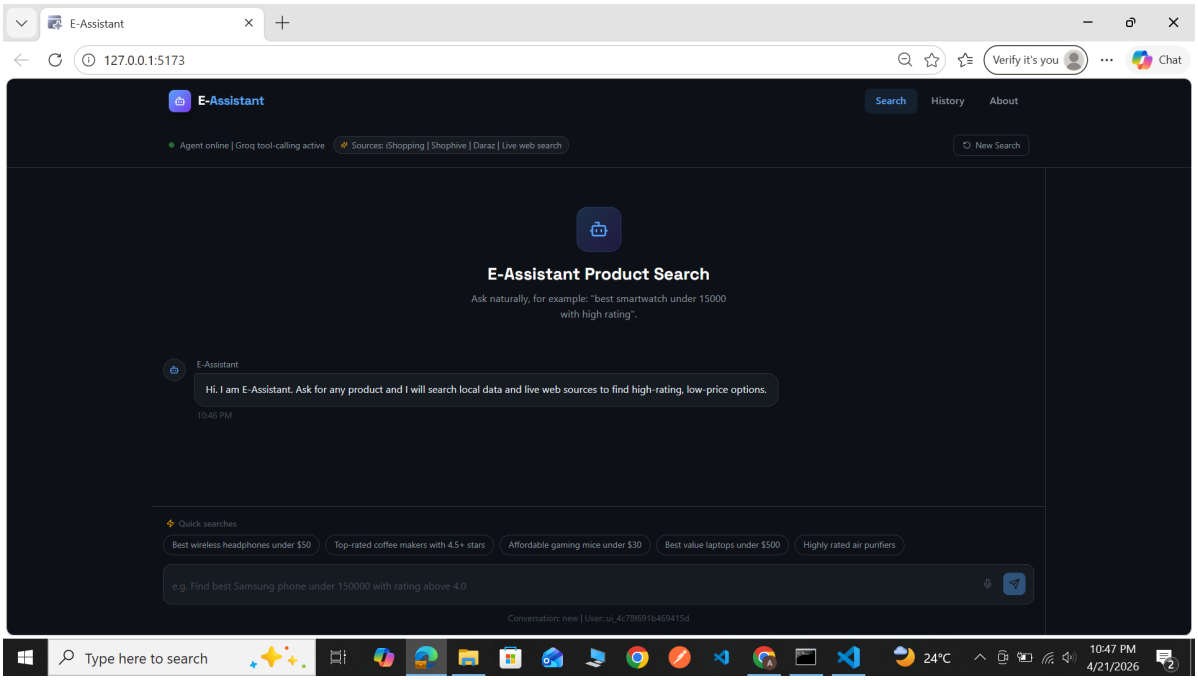
The system combines multiple frameworks across frontend, backend, data, and intelligence layers.

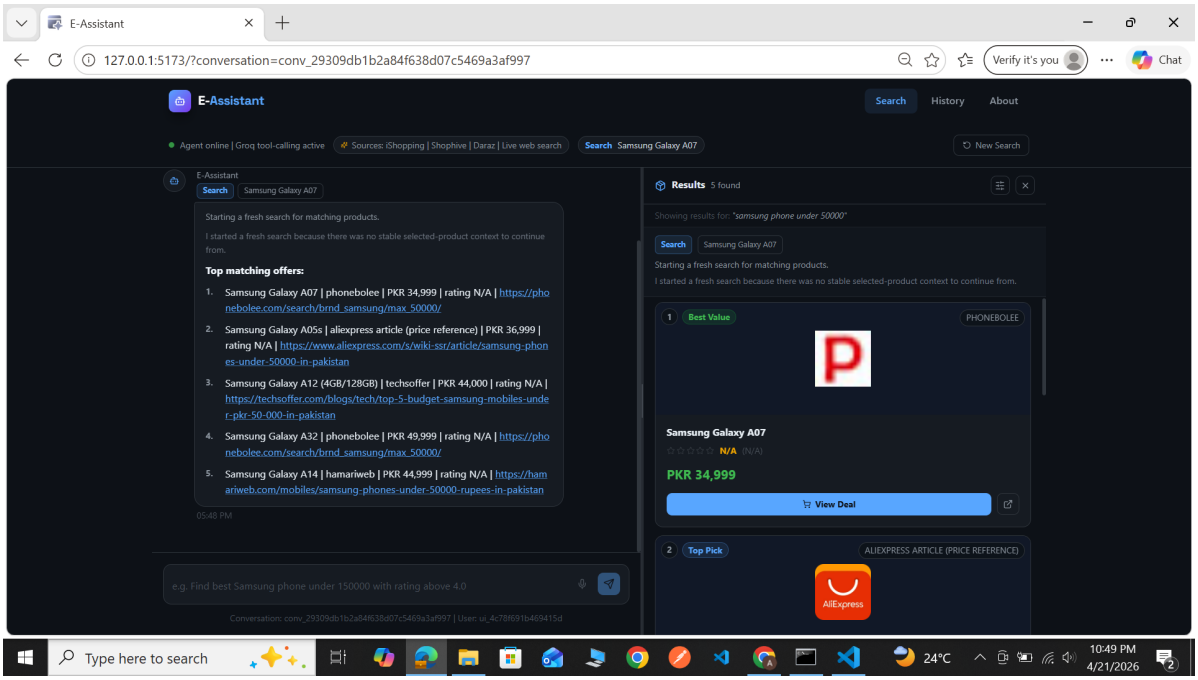
Table 4.3: Tools and technologies used in the implemented product-search system.

Layer	Technology	Purpose in the Project
Frontend	React, TypeScript, Vite, React Router	Search UI, history page, navigation, component-based rendering, and local development server.
Frontend interaction	Framer Motion, Lucide React	Motion effects, UI transitions, and icon-based interface elements.
Backend API	FastAPI, Uvicorn, Pydantic	Request validation, routing, service hosting, and API contracts.
Database	MongoDB, PyMongo	Storage for normalized offers, sessions, turns, interactions, audit logs, and labeling data.
Assistant runtime	Python orchestration layer	Turn planning, clarification logic, follow-up handling, comparison flow, refinement flow, and context persistence.
Search and ranking	Sentence-transformer-style matcher, clustering, reranking, CF utilities	Semantic retrieval support, product grouping, ranking, and personalization blending.
Web extraction	Playwright, Requests, Beautiful Soup	Scraping configured sources and inspecting product pages for live signals.
LLM services	Groq-backed model calls	Query parsing, assistant JSON planning, and live web search support when enabled.
Supplementary UI	Gradio	Optional lightweight demonstration and testing interface.
Deployment	Python runner, Docker, Docker Compose	Local orchestration and containerized service startup.

4.8 User Interface

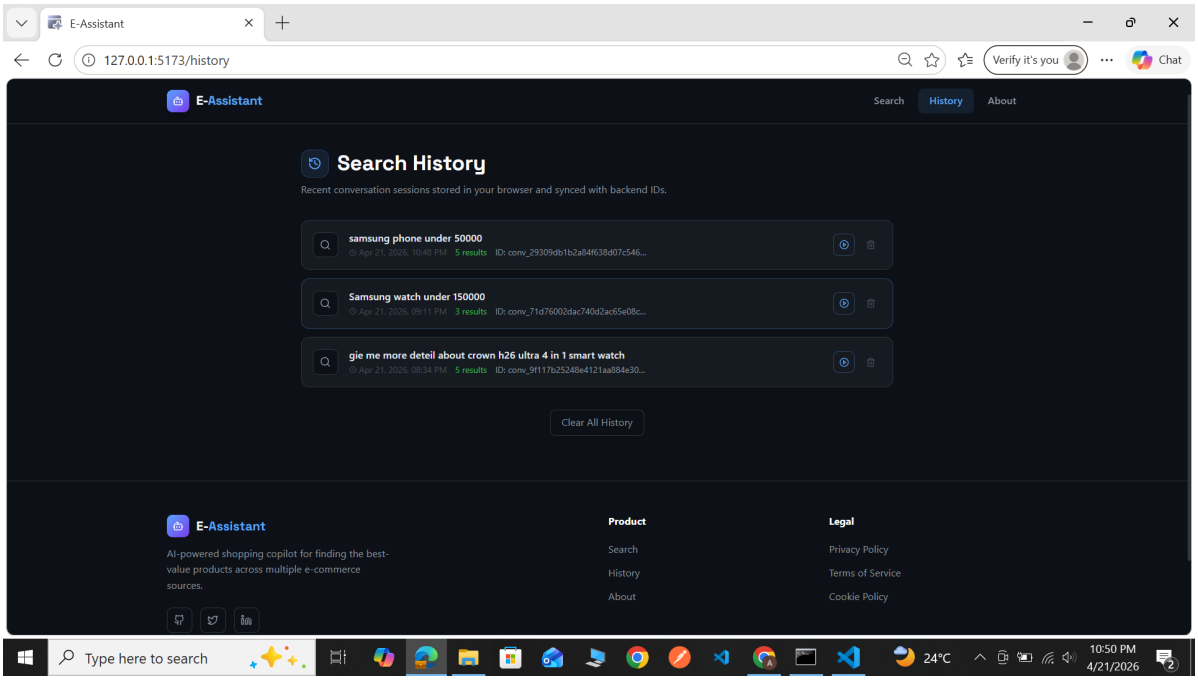
The UI section documents the implemented E-Assistant frontend through screenshots stored in the project's documentation folder. The screenshots cover the dashboard-like landing state, the result-rich search state, the history page, and the about page.

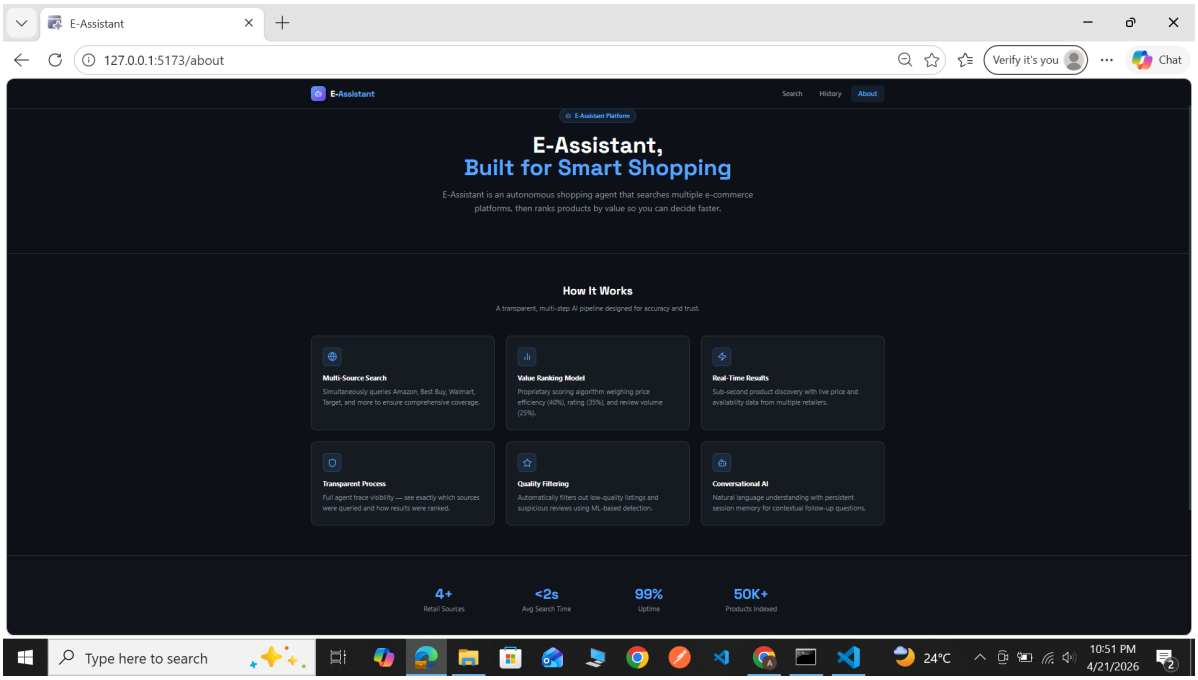

E-Assistant
127.0.0.1:5173
Verify it's you
Chat
E-Assistant
Search
History
About
Agent online | Groq tool-calling active
Sources: iShopping | Shophive | Daraz | Live web search
New Search
E-Assistant Product Search
Ask naturally, for example: "best smartwatch under 15000 with high rating".
E-Assistant
Hi. I am E-Assistant. Ask for any product and I will search local data and live web sources to find high-rating, low-price options.
10:46 PM
Quick searches
Best wireless headphones under $50
Top-rated coffee makers with 4.5+ stars
Affordable gaming mice under $30
Best value laptops under $500
Highly rated air purifiers
e.g. Find best Samsung phone under 150000 with rating above 4.0
Conversation: new | User: ui_4c78f691b469415d
Type here to search
24°C
10:47 PM
4/21/2026


E-Assistant
127.0.0.1:5173/?conversation=conv_29309db1b2a84f638d07c5469a3af997
Verify it's you
Chat
E-Assistant
Search
History
About
Agent online | Groq tool-calling active
Sources: iShopping | Shophive | Daraz | Live web search
Search Samsung Galaxy A07
New Search
E-Assistant
Search
Samsung Galaxy A07
Starting a fresh search for matching products.
I started a fresh search because there was no stable selected-product context to continue from.
Top matching offers:
1. Samsung Galaxy A07 | phonebolee | PKR 34,999 | rating N/A | https://phonebolee.com/search/brnd_samsung/max_50000/
2. Samsung Galaxy A05s | aliexpress article (price reference) | PKR 36,999 | rating N/A | https://www.aliexpress.com/s/wiki-ssr/article/samsung-phones-under-50000-in-pakistan
3. Samsung Galaxy A12 (4GB/128GB) | techsoffer | PKR 44,000 | rating N/A | https://techsoffer.com/blogs/tech/top-5-budget-samsung-mobiles-under-pkr-50-000-in-pakistan
4. Samsung Galaxy A32 | phonebolee | PKR 49,999 | rating N/A | https://phonebolee.com/search/brnd_samsung/max_50000/
5. Samsung Galaxy A14 | hamariweb | PKR 44,999 | rating N/A | https://hamariweb.com/mobiles/samsung-phones-under-50000-rupees-in-pakistan
05:48 PM
Results 5 found
Showing results for: "samsung phone under 50000"
Search
Samsung Galaxy A07
Starting a fresh search for matching products.
I started a fresh search because there was no stable selected-product context to continue from.
1
Best Value
PHONEBOLEE
Samsung Galaxy A07
N/A (N/A)
PKR 34,999
View Deal
2
Top Pick
ALIEXPRESS ARTICLE (PRICE REFERENCE)
AliExpress
e.g. Find best Samsung phone under 150000 with rating above 4.0
Conversation: conv_29309db1b2a84f638d07c5469a3af997 | User: ui_4c78f691b469415d
Type here to search
24°C
10:49 PM
4/21/2026

4.8.1 Main Search and Result Workflow

The first screenshot page shows the search dashboard in both its initial and active states. The initial state emphasizes the branded entry point, quick prompts, and the main search box. The active result state shows how the conversation pane, assistant context chips, and ranked product panel work together once a search has been executed. This reflects the core product workflow of the system.


E-Assistant
127.0.0.1:5173/history
Verify it's you
Chat
E-Assistant
Search
History
About
Search History
Recent conversation sessions stored in your browser and synced with backend IDs.
samsung phone under 50000
Samsung watch under 150000
gie me more deteil about crown h26 ultra 4 in 1 smart watch
Clear All History
E-Assistant
AI-powered shopping copilot for finding the best-value products across multiple e-commerce sources.
Product
Search
History
About
Legal
Privacy Policy
Terms of Service
Cookie Policy
Type here to search
24°C
10:50 PM
4/21/2026


E-Assistant
127.0.0.1:5173/about
Verify it's you
Chat
E-Assistant
Search
History
About
E-Assistant Platform
E-Assistant,
Built for Smart Shopping
E-Assistant is an autonomous shopping agent that searches multiple e-commerce platforms, then ranks products by value so you can decide faster.
How It Works
A transparent, multi-step AI pipeline designed for accuracy and trust.
Multi-Source Search
Value Ranking Model
Real-Time Results
Transparent Process
Quality Filtering
Conversational AI
4+
<2s
99%
50K+
Type here to search
24°C
10:51 PM
4/21/2026

4.8.2 History and About Screens

The second screenshot page shows the history page and the about page. The history page demonstrates session restoration and deletion support for earlier searches. The about page provides a concise explanatory view of the platform's purpose and feature highlights. Together, these screens complete the main user-facing structure of the application.

4.9 Deployment

The project is deployed as a lightweight multi-service application. The primary demonstration path starts a FastAPI backend and a Vite frontend together through the local runner. MongoDB provides the persistent data layer, and an optional Docker Compose stack is available for containerized execution.

The local deployment path is centered around ‘run_all.py‘. This script resolves the frontend toolchain, optionally installs dependencies, starts the API, waits for the health endpoint to become ready, and then launches the React frontend with the appropriate proxy target. This makes local project evaluation straightforward.

The container deployment path is defined through ‘docker-compose.yml‘. It provisions MongoDB, builds the API container, and builds the frontend container. The web service exposes the interface through port 3000 while the API remains available on port 8000. This is sufficient for demonstration and review even though it is not intended as a production-scale deployment model.

Table 4.4: Deployment environment summary.

Component	Runtime	Deployment Role	Purpose
React Frontend	Node.js / Vite or Nginx container	Presentation service	Provides search, history, about, and ranked-result UI.
FastAPI Backend	Python / Uvicorn	Intelligence and API service	Exposes assistant, search, recommendation, history, metrics, and labeling endpoints.
MongoDB	MongoDB service	Persistence layer	Stores offers, sessions, turns, interactions, audit logs, and related project data.
Gradio Interface	Python / Gradio	Supplementary interface	Provides an alternative lightweight demonstration entry point.
Docker Compose	Docker runtime	Deployment orchestration	Starts web, API, and MongoDB services together for containerized execution.

Table 4.5: Deployment components and configuration detail.

Deployment Item	Detail
Environment configuration	‘.env‘ defines Mongo URI, API keys, model settings, rate limits, and live search behavior.
Local startup runner	‘run_all.py‘ starts the API first, waits for health readiness, then launches the frontend with the correct proxy target.
API startup	The backend is served through ‘uvicorn src.api.app:app‘ with configurable host and port.
Frontend startup	The React app runs through ‘npm run dev‘ for development or through the built container path in Docker deployment.
Mongo dependency	MongoDB is required for offer storage, conversation persistence, interactions, and audit records.
Operational controls	Authorization mode, rate-limiter backend, metrics exposure, and live web behavior are all configuration-driven.
Container deployment	Docker Compose provisions Mongo, API, and web services with health dependencies between them.

Operationally, the system is suitable for classroom evaluation, local demonstration, and controlled project testing. Before any broader public deployment, the project would still require stronger operational hardening, source-governance controls, secret handling, and more formal monitoring of scraping and live inspection reliability.

4.10 Chapter Summary

This chapter documented the implemented system from design and architecture perspectives. It defined the major modules, explained the layered runtime structure, and provided diagrams for workflow, actors, classes, sequences, and states. It also described the data design, human interface, core algorithms, toolchain, screenshots, and deployment model. Together, these sections show how the project functions as a complete AI-powered product-search application.

Bibliography

[1] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019.

[2] Sebastian Ramirez. FastAPI documentation. https://fastapi.tiangolo.com/

[3] MongoDB. MongoDB documentation. https://www.mongodb.com/docs/

[4] React. React documentation. https://react.dev/

[5] Vite. Vite documentation. https://vite.dev/

[6] Microsoft. Playwright documentation. https://playwright.dev/

[7] Beautiful Soup. Beautiful Soup documentation. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[8] Gradio. Gradio documentation. https://www.gradio.app/docs